String matching

Part VII Selected Topics

Matching reuses known equal prefixes and suffixes as state. We derive finite automata and KMP from character comparison and modular hashing, proving the failure chain, correctness, and total linear cost. Prerequisites are pattern prefixes, hashing, and finite automata.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover naive matching, Rabin-Karp, finite-automaton matching, and Knuth-Morris-Pratt. The goals are to define valid shifts and borders, compare these four methods, and prove the prefix function and KMP time bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Indices, shifts, and boundary cases

Text T has length n, pattern P has length m, and characters belong to finite alphabet Σ. Equality costs $O(1)$. Character indices start at 1, but shifts start at 0. A valid shift satisfies $0 \le s \le n - m$ and $P[j] = T[s + j]$ for **every** $j = 1, \ldots, m$. Output all shifts, including overlaps.

Write $P_k = P[1..k]$, with $P_0 = \epsilon$. A proper prefix or suffix cannot equal the whole string. A border is both a prefix and a suffix.

For $m = 0$, the specification outputs all $n + 1$ boundaries $0, \ldots, n$. For $m > n$, output nothing. Handle these before the main algorithms, which assume $1 \le m \le n$, so they never read a nonexistent $P[1]$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Naive matching checks every shift

```
for s=0..n-m:
    j=1
    while j<=m and P[j]==T[s+j]: j++
    if j==m+1: output s
```

Before each inner comparison, $P[1..j-1] = T[s+1..s+j-1]$. Equality extends the invariant. A mismatch invalidates this shift. Reaching $j = m + 1$ means every character agrees. The outer loop enumerates exactly the possible shifts, proving both soundness and completeness.

There are $n - m + 1$ windows and at most m comparisons each. The worst case is $\Theta((n - m + 1)m)$, attained by $T = a^n, P = a^m$. Many true matches alone can force this cost. Extra workspace is $O(1)$, excluding output.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

All windows of one text

For $T = \texttt{ababaca}$, $P = \texttt{aba}$, stop at the first mismatch.

Shift	Window	Comparisons in order	Count	Result
0	aba	a=a, b=b, a=a	3	Output 0
1	bab	a≠b	1	Reject
2	aba	a=a, b=b, a=a	3	Output 2
3	bac	a≠b	1	Reject
4	aca	a=a, b≠c	2	Reject

There are ten comparisons. Matches at shifts 0 and 2 share text position 3 and are both valid. Naive matching rechecks alignments. KMP uses known equal sections to rule out impossible shifts.

Rabin-Karp rolling identity

Encode characters injectively as $0, \ldots, d-1$. For a length-m string, $H(X) = \sum_{j=1}^{m} X[j] d^{m-j}$. Arithmetic on an arbitrary m-digit integer is not constant time. Store $h_q(X) = H(X) \bmod q$, $q \geq 2$, and verify characters whenever hashes agree.

Let $p = h_q(P), t_s = h_q(T[s+1..s+m]), h = d^{m-1} \bmod q$. Removing the highest digit, shifting, and adding a digit gives

$$t_{s+1} = \big(d(t_s - T[s+1]h) + T[s+m+1]\big) \bmod q.$$

The identity holds for every $q \geq 2$. Primality helps some randomized analyses but does not by itself give collision probability $1/q$ for fixed strings. Normalize negative remainders to $[0, q-1]$ and prevent overflow in intermediate products.

Hashes reject; characters accept

```
h=1; p=0; t=0
for j=1..m-1: h=d*h mod q
for j=1..m:
    p=(d*p+P[j]) mod q
    t=(d*t+T[j]) mod q
for s=0..n-m:
    if p==t and P[1..m]==T[s+1..s+m]: output s
    if s<n-m:
        t=(d*(t-T[s+1]*h)+T[s+m+1]) mod q
```

Horner evaluation and the rolling identity inductively maintain the current window hash. Equal strings have equal hashes, so no match is lost. Different hashes reject. Equal hashes trigger exact comparison, preventing false reports. Collisions affect running time rather than correctness in this verified version. Omitting verification creates a different algorithm that can report false matches.

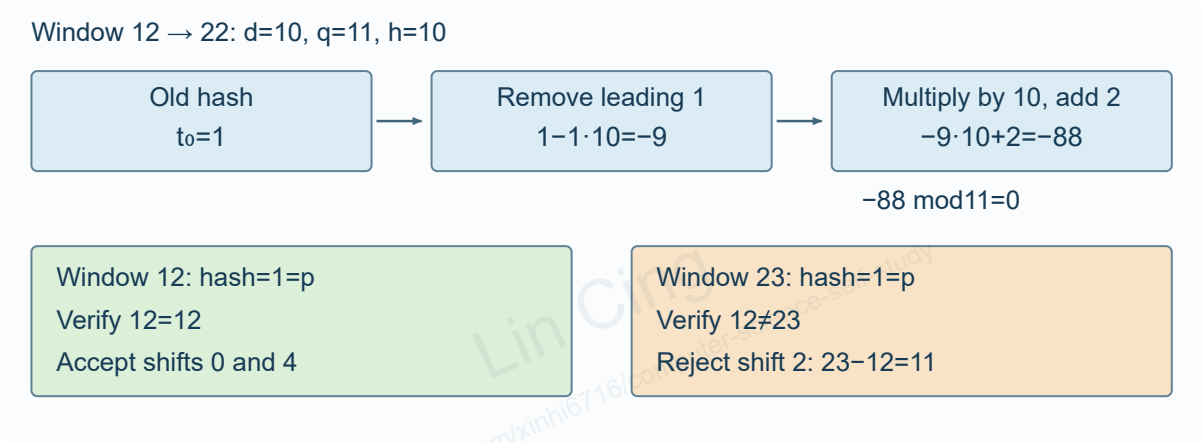
Match, collision, match

Take $T = 122312, P = 12, d = 10, q = 11$. Then $h = 10, p = 1$.

Shift	Window	Hash	Next hash calculation	Verification
0	12	1	$(10(1 - 1 \cdot 10) + 2) \bmod 11 = 0$	Match, output 0
1	22	0	$(10(0 - 2 \cdot 10) + 3) \bmod 11 = 1$	Different hash, skip
2	23	1	$(10(1 - 2 \cdot 10) + 1) \bmod 11 = 9$	$12 \neq 23$, reject collision
3	31	9	$(10(9 - 3 \cdot 10) + 2) \bmod 11 = 1$	Different hash, skip
4	12	1	Final window, read no new character	Match, output 4

A spurious hit has equal hashes but different strings. Here $23 - 12 = 11$ vanishes modulo the prime modulus. Prime moduli still permit collisions.

Match, collision, match (continued)



True and spurious hits both cost time

In the word-RAM model, characters, q, and modular products fit in constant-cost word operations, or constant-cost modular multiplication is available. The intermediate $d(t - \text{out} \cdot h)$ must also fit. Normalizing the difference before multiplication keeps that product below dq. Arbitrary-precision implementations must include bit costs.

Preprocessing takes $\Theta(m)$, and rolling takes $\Theta(n - m + 1)$. With z true matches and f spurious hits, verification takes at most m comparisons per hit, giving $O(n + m + m(z + f))$.

Either z or f can be $\Theta(n - m + 1)$, so worst-case time remains $\Theta((n - m + 1)m)$. All-a inputs have only true hits and still repeat full verification. Output alone costs $\Omega(z)$, while this version spends another m comparisons per true hit.

A proved random-base hash bound

Fix prime $q > d - 1$, so distinct character codes remain distinct in $\mathbb{F}_q$. **After fixing the input**, choose a uniform base $b \in \mathbb{F}_q$, independently of the input. Hash with $\sum_j X[j] b^{m-j} \bmod q$. Replace d with b in the rolling identity. The identity also handles $b = 0$, with $h = b^0 = 1$ when $m = 1$.

For fixed distinct strings X, Y, their difference polynomial is nonzero and has degree at most $m - 1$. A nonzero field polynomial has at most its degree many roots. At a root, factor out its linear factor and induct on degree. Thus

$$\Pr[h_b(X) = h_b(Y)] \le \min(1, (m-1)/q).$$

Sum the expectations of collision indicators over nonmatching windows. Independence between windows is unnecessary, and $E[f] \le (n - m + 1 - z)(m - 1)/q$. A loose expected-time bound is $O(n + m + mz + mnm/q)$. A large enough modulus reduces spurious-hit cost but not the mz cost of true matches. Its bit length and modular arithmetic costs must still be counted.

Finite-automaton state

A finite automaton consists of $(Q, q_0, A, \Sigma, \delta)$, a finite state set, initial state, accepting set, alphabet, and transition function for every state-character pair. For matching use $Q = \{0, \dots, m\}$, $q_0 = 0$, $A = \{m\}$.

Define $\sigma(x) = \max\{k \le m : P_k \text{ is a suffix of } x\}$. The empty string is always a suffix, so the maximum exists. The state $q = \sigma(x)$ is the longest pattern prefix currently available for extension, rather than the number of completed matches.

Set $\delta(q, a) = \sigma(P_q a)$. Read one character and take one transition. State m reports the shift ending at that position. Accepting states still have outgoing transitions, allowing overlapping matches.

Why the longest prefix is sufficient

We prove $\delta(\sigma(x), a) = \sigma(xa)$. Let $q = \sigma(x), r = \sigma(xa)$. If $r > 0$, remove the last character from its match. Then P_{r-1} is a suffix of x, so $r - 1 \leq q$. Since both are suffixes of x, the shorter P_{r-1} is a suffix of P_q. Appending a makes P_r a suffix of $P_q a$, hence $r \leq \sigma(P_q a)$. This inequality also holds for $r = 0$.

Conversely, $P_q a$ is a suffix of xa. Every suffix of $P_q a$ that is a pattern prefix is therefore a suffix of xa. Thus $\sigma(P_q a) \leq \sigma(xa) = r$, proving equality.

Starting with $\sigma(\epsilon) = 0$, induction gives $q = \sigma(T[1..i])$ after each transition. State m holds exactly when P is a suffix of that text prefix, equivalent to a match at shift $i - m$. This proves both directions of acceptance.

Complete transition table and overlap

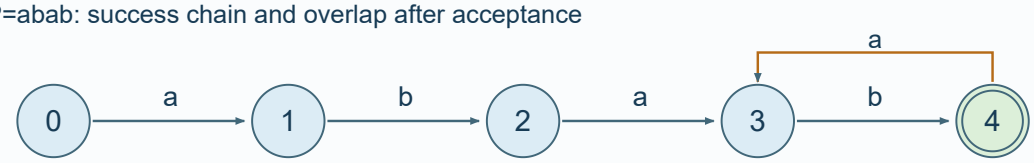
For $P = \texttt{abab}, \Sigma = \{a, b, c\}$:

q	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	3	0	0

Scanning `ababab` gives states 1,2,3,4,3,4, reporting shifts 0 and 2 at positions 4 and 6. The transition $\delta(4, a) = 3$ uses suffix `aba` of `ababa`. Resetting to zero after a match would lose the second occurrence.

Complete transition table and overlap (continued)

P=abab: success chain and overlap after acceptance



The full transition table is in the notes. Only the success chain and δ(4,a)=3 are shown.

T[i]	a	b	a	b	a	b
State	1	2	3	4	3	4

Accept at positions 4 and 6: shifts 0 and 2. Do not reset after a match.

Table construction versus scanning

A direct construction tries candidate suffix lengths from $\min(m, q+1)$ downward for each q, a, comparing characters. It can take $O(m^2)$ per cell and $O(m^3|\Sigma|)$ overall. It is correct but slow. Scanning is still $\Theta(n)$.

Using the prefix function π below, construct in increasing q:

```
for a in Sigma: delta(0,a) = (1 if a==P[1] else 0)
for q=1..m:
    for a in Sigma:
        if q<m and a==P[q+1]: delta(q,a)=q+1
        else: delta(q,a)=delta(pi[q],a)
```

After mismatch, the next candidate is a proper border of P_q. The longest has length $\pi[q]$, and all shorter candidates are suffixes of it. Reuse its transition on the same character. Since $\pi[q] < q$, dependencies are already computed. Computing π takes $\Theta(m)$. The table takes $\Theta(m|\Sigma|)$ time and space. Constant-time character lookup assumes constant-time alphabet indexing.

The prefix function

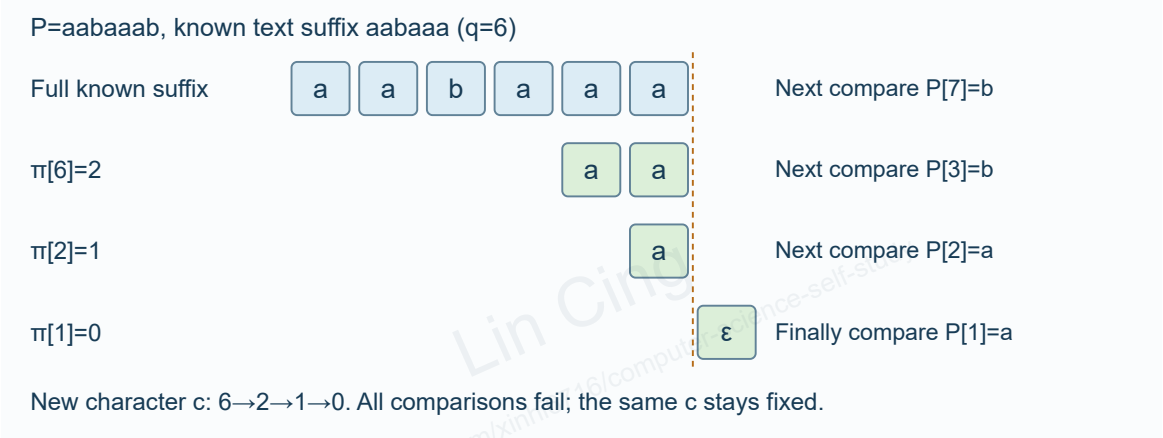
Define $\pi[q] = \max\{k < q : P_k \text{ is a suffix of } P_q\}$, for $q = 1, \ldots, m$. Thus $\pi[1] = 0$ and $\pi[q] < q$. It depends only on the pattern.

For aabaaab, $\pi = [0, 1, 0, 1, 2, 2, 3]$. The proper borders of $P_6 =$ **aabaaa** are aa, a, and the empty string. The chain is $6 \to 2 \to 1 \to 0$. Length 6 is the starting state, not a proper border.

KMP stores no full transition table. On a mismatch, follow π to a shorter candidate while keeping the new text character fixed. Previously read text need not be read again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The prefix function (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The failure chain contains every candidate

Let $B(q)$ be all proper border lengths of P_q, and let $b = \pi[q] > 0$. For every $k < b$,

$$k \in B(q) \quad \iff \quad k \in B(b).$$

Since P_b is a suffix of P_q, their length-k suffixes coincide. Their length-k prefixes also coincide with P_k. Thus the prefix-suffix equality is identical in both directions. No border has $b < k < q$, by maximality. If $b = 0$, no positive proper border exists.

Hence $B(q) = \{b\} \cup B(b)$. Induction on strictly decreasing lengths proves that $\pi[q], \pi[\pi[q]], \ldots, 0$ lists every proper border. This completeness justifies fallback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Construct the prefix function

```
pi[1]=0; k=0
for i=2..m:
    while k>0 and P[k+1]!=P[i]: k=pi[k]
    if P[k+1]==P[i]: k++
    pi[i]=k
```

Before processing i, $k = \pi[i-1]$. If P_i has a positive proper border of length r, deleting its last character leaves a proper border of P_{i-1} of length $r-1$. Conversely, a border of that length extends when $P[r] = P[i]$.

Start at the largest candidate $\pi[i-1]$ and follow the complete decreasing failure chain. The first extendible candidate gives the largest legal r. At zero, unequal characters give answer zero. Only smaller, already-correct prefix values are used, proving induction. Since $\pi[i-1] \le i-2$, the entire P_i cannot be mistaken for its own proper border.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every prefix-table step

For `aabaaab`:

i	Character	Initial k	Comparison and fallback	pi[i]
1	a	None	A single character has no nonempty proper border	0
2	a	0	P[1]=a, extend	1
3	b	1	P[2]=a≠b; 1→0; P[1] also differs	0
4	a	0	P[1]=a	1
5	a	1	P[2]=a	2
6	a	2	P[3]=b≠a; 2→1; P[2]=a	2
7	b	2	P[3]=b	3

The value $\pi[6] = 2$, not 3, because suffix `aaa` differs from prefix `aab`. Fallback keeps known possible borders rather than always subtracting one.

KMP scan and the state after a match

```
pi=prefix_function(P); q=0
for i=1..n:
    while q>0 and P[q+1]!=T[i]: q=pi[q]
    if P[q+1]==T[i]: q++
    if q==m:
        output i-m
        q=pi[m]
```

At the acceptance check, $q = \sigma(T[1..i])$, as in the automaton. If $q < m$, retain it. At $q = m$, output first, then retain $\pi[m]$, the longest proper pattern prefix that is a suffix of the read text. A new match only needs a proper border, since a full length-m prefix cannot extend to length $m + 1$.

Every shorter candidate after mismatch lies on the failure chain. The first that extends with $T[i]$ gives exactly the automaton transition. If none extends, use zero. This proves completeness and ensures $q < m$ before accessing $P[q + 1]$.

Overlap after a full match

For $P = \texttt{aabaaab}$, $\pi[7] = 3$. Scan $T = \texttt{aabaaabaaab}$.

i	Character	q before comparison	q after extension	q retained after output
1	a	0	1	1
2	a	1	2	2
3	b	2	3	3
4	a	3	4	4
5	a	4	5	5
6	a	5	6	6
7	b	6	7, output 0	3
8	a	3	4	4
9	a	4	5	5

Overlap after a full match (continued)

i	Character	q before comparison	q after extension	q retained after output
10	a	5	6	6
11	b	6	7, output 4	3

The matches share $\texttt{aab}$ at positions 5 through 7. Resetting to zero discards that known equality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Many comparisons for one character, linear total

From state 6 on new character c, compare against $P[7] = b, P[3] = b, P[2] = a, P[1] = a$. All differ. Follow $6 \to 2 \to 1 \to 0$ and retain zero. The same c is compared repeatedly, but the text index never retreats.

During the scan, q starts at zero and increases by at most one per character, for total increase at most n. Every while-body iteration decreases it by at least one. Post-match fallback also only decreases it, and q stays nonnegative. Thus there are at most n while iterations, plus constant work per character and output. Scanning takes $\Theta(n)$. The same accounting gives at most $m - 1$ increases and while iterations in prefix construction, hence $\Theta(m)$.

Total time is $\Theta(n + m)$, with $\Theta(m)$ workspace excluding output. This is a sequence bound, not a worst-case constant bound for each character. A full automaton table provides that per-character bound at a higher space cost.

Assumptions of the four methods

Method	Preprocessing	Worst-case scan	Pattern workspace
Naive	None	$\Theta((n - m + 1)m)$	$O(1)$
Verified Rabin-Karp	$\Theta(m)$	$\Theta((n - m + 1)m)$	$O(1)$
Full automaton using pi	$\Theta(m\|\Sigma\|)$	$\Theta(n)$	$\Theta(m\|\Sigma\|)$
KMP	$\Theta(m)$	$\Theta(n)$	$\Theta(m)$

The table assumes $1 \leq m \leq n$, constant-cost characters, and word modular arithmetic. Preprocessing is separate. Fast expected Rabin-Karp requires explicit randomization and collision conditions. KMP requires neither random input nor a random pattern. Unicode implementations must choose bytes, code points, or grapheme clusters as the matching units. Different encoded character sequences are not automatically identical strings.

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 32.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study